Validation System Documentation

This directory contains centralized, reusable validation schemas for the Gema backend API.

Structure

```
validators/
├── common.validator.ts    # Reusable validation helpers (MongoDB IDs, pagination, etc.)
├── auth.validator.ts      # Authentication-related validations
├── user.validator.ts      # User profile and account validations
├── event.validator.ts     # Event creation and management validations
├── order.validator.ts     # Order and booking validations
└── README.md             # This file
```

Usage Examples

Basic Usage in Routes

```
import { Router } from 'express';
import { validate } from '../middleware';
import { validateRegistration, validateLogin } from '../validators/auth.validator';
import * as authController from '../controllers/auth.controller';

const router = Router();

// Before: Inline validation (verbose, repetitive)
router.post('/register', [
  body('email').trim().notEmpty().isEmail(),
  body('password').notEmpty().isLength({ min: 8 }),
  // ... many more lines
], validate, authController.register);

// After: Centralized validation (clean, reusable)
router.post('/register', validateRegistration, validate, authController.register);
router.post('/login', validateLogin, validate, authController.login);
```

Combining Multiple Validators

```
import { validateMongoId, validatePagination } from '../validators/common.validator';
import { validateEventSearch } from '../validators/event.validator';

// Combine common validators with specific ones
router.get(
  '/events/:id',
  validateMongoId('id', 'param'),
  validate,
  eventController.getEvent
);

// Combine multiple validator arrays
router.get(
  '/events',
  validateEventSearch, // Includes pagination, sorting, filtering
  validate,
  eventController.listEvents
);
```

Using Common Validators

The `common.validator.ts` file provides frequently-used validators:

```
import {
  validateMongoId,      // MongoDB ObjectId validation
  validatePagination,   // page, limit query params
  validateSort,         // sortBy, sortOrder query params
  validateDateRange,    // startDate, endDate query params
  validatePriceRange,   // minPrice, maxPrice query params
  validateSearch,       // search query param with XSS protection
  validateEmail,        // Email validation with normalization
  validatePhone,        // International phone number validation
  validateEnum,         // Enum/allowed values validation
} from '../validators/common.validator';

// Example: Event search endpoint
router.get('/events', [
  ...validatePagination,
  ...validateSort(['title', 'price', 'createdAt']),
  ...validateDateRange,
  ...validatePriceRange,
], validate, eventController.search);
```

Available Validators

Common Validators (`common.validator.ts`)

Validator	Parameters	Description
`validateMongoId()`	field, location	MongoDB ObjectId validation
`validatePagination`	-	page (min: 1), limit (1-100)
`validateSort()`	allowedFields	sortBy, sortOrder validation
`validateDateRange`	-	startDate, endDate with logic check
`validatePriceRange`	-	minPrice, maxPrice with logic check
`validateSearch`	-	Search query with XSS protection
`validateEmail()`	field, required	Email with normalization
`validatePhone()`	field, required	International phone format
`validateUrl()`	field, required	URL validation
`validateArray()`	field, min, max	Array length validation
`validateStringLength()`	field, min, max, required	String length validation
`validateNumericRange()`	field, min, max, required	Numeric range validation
`validateEnum()`	field, values, required	Enum validation
`sanitizeHtml()`	field	XSS protection for HTML
`validateCoordinates`	-	Latitude/longitude validation

Auth Validators (`auth.validator.ts`)

- `validateRegistration` - User registration
- `validateLogin` - User login
- `validateRefreshToken` - Token refresh
- `validateChangePassword` - Password change
- `validateForgotPassword` - Password reset request
- `validateResetPassword` - Password reset confirmation
- `validateEmailVerification` - Email OTP verification
- `validateResendVerification` - Resend verification email
- `validateFirebaseAuth` - Firebase authentication
- `validate2FASetup` - Two-factor auth setup
- `validate2FAVerification` - 2FA token verification

User Validators (`user.validator.ts`)

- `validateProfileUpdate` - Update user profile
- `validateAddress` - Add/update address
- `validateAddressIndex` - Validate address array index
- `validateAvatarUpdate` - Update avatar URL
- `validateBusinessHours` - Vendor business hours
- `validateSocialMedia` - Social media links
- `validateVendorPaymentSettings` - Vendor payment settings
- `validateUserStatusUpdate` - User status (admin)
- `validateUserRoleUpdate` - User role (admin)

Event Validators (`event.validator.ts`)

- `validateCreateEvent` - Create new event
- `validateUpdateEvent` - Update existing event
- `validateEventFAQ` - Event FAQs
- `validateEventSEO` - SEO metadata
- `validateEventSearch` - Event search/filtering
- `validateEventApproval` - Event approval (admin)

Order Validators (`order.validator.ts`)

- `validateCreateOrder` - Create new order
- `validateParticipants` - Participant information
- `validateUpdateOrderStatus` - Update order status
- `validateUpdatePaymentStatus` - Update payment status
- `validateRefundOrder` - Process refund
- `validateCheckIn` - Check-in for event
- `validateOrderSearch` - Order search/filtering

Benefits

1. **Code Reusability**: Define validation rules once, use everywhere
2. **Consistency**: Same validation logic across all endpoints
3. **Maintainability**: Update validation in one place
4. **Security**: Built-in XSS protection, input sanitization
5. **Type Safety**: TypeScript support for better autocomplete
6. **Cleaner Routes**: Reduced code from ~30 lines to 1 line per endpoint

Migration Guide

To migrate existing routes to use the new validation system:

1. Import the appropriate validators:

```
import { validateLogin } from '../validators/auth.validator';
```

2. Replace inline validation arrays:

```
// Before
router.post('/login', [body('email')...], validate, controller.login);

// After
router.post('/login', validateLogin, validate, controller.login);
```

3. Test the endpoint to ensure validation works correctly

Adding New Validators

When adding new validators:

1. Use existing validators in `common.validator.ts` as building blocks
2. Keep validators focused and reusable
3. Add JSDoc comments explaining parameters and usage
4. Export as named export for tree-shaking
5. Update this README with the new validator

Example:

```
/**
 * Validate coupon code
 */
export const validateCouponCode = [
  body('code')
    .trim()
    .notEmpty()
    .withMessage('Coupon code is required')
    .isLength({ min: 3, max: 20 })
    .withMessage('Coupon code must be between 3 and 20 characters')
    .isAlphanumeric()
    .withMessage('Coupon code must contain only letters and numbers')
    .toUpperCase(),
];
```

Security Features

All validators include:

- **XSS Protection**: `.escape()` for user input
- **SQL Injection Protection**: MongoDB parameterized queries
- **Input Sanitization**: `.trim()`, `.toLowerCase()` where appropriate
- **Type Coercion**: `.toInt()`, `.toFloat()`, `.toBoolean()` for type safety
- **Length Limits**: Prevent DoS attacks with max lengths
- **Enum Validation**: Whitelist-based validation for allowed values